Reinforcement Learning

From trial-and-error to optimal control: bandits, MDPs, Bellman equations, Q-learning — and the road to deep RL

1 · What makes RL different

Supervised learning gets the correct answer for every input; unsupervised learning gets no feedback at all. Reinforcement learning sits apart: an **agent** interacts with an **environment**, and the only feedback is a scalar **reward** — *evaluative* (it scores the action you took) rather than *instructive* (it never names the right one). Two consequences shape the whole field:

- **Delayed consequences.** Actions pay off many steps later — which of the forty moves won the chess game? This is the credit-assignment problem.
- **The agent shapes its own data.** Behaviour determines what is observed next; the data is non-i.i.d. and depends on the policy itself.

The reward hypothesis (Sutton & Barto): all goals can be described as maximisation of expected cumulative reward. Chess: +1 win, −1 loss, 0 otherwise. Robot: + forward velocity, − energy, − falling. RLHF: a reward model's score of a human-preferred response. Reward defines *what* to achieve, never *how*.

2 · Warm-up: the k-armed bandit

One state, k actions, unknown mean payouts — RL with the sequential part removed. It isolates **exploration vs exploitation** and runs daily in A/B tests and ad serving.

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a], \qquad \text{maximise } \sum_{t=1}^{T} R_t$$

Sub-goal: estimate $q_*(a)$ while still earning reward — estimation and control at once.

$$Q_{n+1} = Q_n + \tfrac{1}{n}[R_n - Q_n]$$

The sample average, computed incrementally — no need to store old rewards. A 1/n step gives the true average (stationary arms); a constant α gives a recency-weighted average that tracks drifting arms.

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \cdot [\text{Target} - \text{OldEstimate}]$$

The master template. Every update in these notes is this line with a different Target.

ε-greedy: with probability 1−ε exploit (argmax Q), with probability ε explore (random arm). Pure greedy locks onto the first arm that looks good and never corrects the others. Beyond ε-greedy: optimistic initialisation, the UCB bonus $c\sqrt{\ln t / N(a)}$, Thompson sampling, decaying ε.

3 · The Markov Decision Process

Add state: the world evolves under your actions. The formal object is $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$: states, actions, transition dynamics $P(s'|s,a)$, expected rewards $r(s,a)$, and the discount γ. **P and R together are the model** — known model → planning; unknown → learning.

$$P(S_{t+1}, R_{t+1} \mid S_t, A_t) = P(S_{t+1}, R_{t+1} \mid S_1, A_1, \ldots, S_t, A_t)$$

Markov property: the future is independent of the past, given the present. Chess position ✓, inventory level ✓; a single camera frame ✗ (velocity invisible — stack 4 frames, the DQN trick), poker ✗ (hidden cards — a POMDP, needs belief states). Fix: engineer the state until it is approximately Markov.

Policy: $\pi(a|s) = P(A_t = a \mid S_t = s)$ — the agent's behaviour made explicit. Deterministic policies suffice for optimal play in finite MDPs; stochastic ones are needed while exploring and under partial observability.

4 · Return, discount, value

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \qquad G_t = R_{t+1} + \gamma G_{t+1}$$

The recursion comes from factoring γ out of every term after the first — remember it; it becomes the Bellman equation. Since $\sum \gamma^k = 1/(1-\gamma)$ (geometric series: S − γS = 1), γ sets an **effective horizon ≈ 1/(1−γ)** steps. γ = 0: myopic; γ → 1: far-sighted; γ = 0.99 ≈ a 100-step horizon.

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s], \qquad Q^{\pi}(s,a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

V scores a state, Q an action in a state; $V^{\pi}(s) = \sum_a \pi(a|s) Q^{\pi}(s,a)$. Q matters more in practice: greedy on Q needs no model, while acting on V alone still requires $P(s'|s,a)$.

5 · The Bellman equations

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) \left[r(s,a) + \gamma V^{\pi}(s') \right]$$

Value = expected one-step reward + discounted value of the next state. Consistency, not magic: $|\mathcal{S}|$ linear equations in $|\mathcal{S}|$ unknowns — with a known model, prediction is linear algebra.

$$Q^*(s,a) = r(s,a) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} Q^*(s',a')$$

Bellman optimality — the most important line in these notes. Replace the policy average with a max. Then $V^* = \max_a Q^*$ and $\pi^*(s) = \arg\max_a Q^*(s,a)$: greedy on Q* is globally optimal. The max makes the system nonlinear — no closed form; solve iteratively. Every finite MDP has a deterministic optimal policy.

6 · Three ways to solve an MDP

	DYNAMIC PROGRAMMING	MONTE CARLO	TEMPORAL DIFFERENCE
Needs a model?	yes — P and R	no — just episodes	no — single steps
When it updates	full sweeps over all states	end of each episode	every step, online
Target	exact expectation	actual return G	R + γV(s′) — bootstrap
Bias / variance	exact (given model)	unbiased · high variance	biased · low variance
Continuing tasks	yes	no — needs episodes	yes

$$V_{k+1}(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V_k(s') \right]$$

Value iteration — repeat the optimality backup until it stops changing. It works because the backup is a contraction: $\|V_{k+1} - V^*\| \le \gamma \|V_k - V^*\|$ — geometric convergence from any start. This is planning, not learning.

Why we can't always plan — two curses. Modelling: $P(s'|s,a)$ is unknown for robots, markets, users, and model errors compound. Dimensionality: DP sweeps all states, and Go has $\approx 10^{170}$. Escapes: **sample** (MC, TD) and **generalise** (function approximation).

$$\text{MC: } V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)] \qquad \text{TD(0): } V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{R_{t+1} + \gamma V(S_{t+1}) - V(S_t)}_{\delta_t,\ \text{TD error}}]$$

MC waits for the real return — unbiased, high variance, episodic only. TD bootstraps on its own estimate — online, low variance, some bias. δ is the "surprise"; dopamine neurons encode exactly this signal. n-step returns and TD(λ) interpolate between the two.

7 · Control: SARSA and Q-learning

$$\textbf{SARSA (on-policy):}\quad Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma Q(S',A') - Q(S,A)\right]$$

A′ is the action the policy actually takes in S′ — the quintuple (S, A, R, S′, A′) names the algorithm. It evaluates the ε-greedy policy it is running, so exploration noise is priced into the values. Converges to Q* under GLIE (explore forever, greedy in the limit) with Robbins–Monro step sizes.

$$\textbf{Q-learning (off-policy, Watkins 1989):}\quad Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_{a'} Q(S',a') - Q(S,A)\right]$$

Behaviour stays ε-greedy; the max-target learns about pure greedy play — exploration and optimality decoupled. Watch out: the max causes overestimation bias (fix: Double Q-learning), and off-policy learning can diverge with function approximation.

Worked example: the 4×4 gridworld

Goal G pays +10 and ends the episode; every other step costs −1. Deterministic moves, walls bounce back. γ = 0.9, α = 0.5, ε = 0.2, Q = 0 initially. Let A be the cell left of G, B the cell left of A.

```
① A→G: r = +10, terminal ⇒ max Q(G,·) = 0.   Q(A,→) ← 0 + 0.5·[10 + 0.9·0 − 0] = 5.0
② B→A: r = −1, max Q(A,·) = 5.0.   Q(B,→) ← 0 + 0.5·[−1 + 0.9·5.0 − 0] = 1.75
Each later visit to A halves the remaining error: 5 → 7.5 → 8.75 → 9.38 → … → 10.
```

Value leaks backwards from the goal, one bootstrap per visit. After convergence (values with greedy arrows):

6.2 →	8.0 →	10 →	G +10
4.6 →	6.2 →	8.0 ↗	10 ↑
3.1 →	4.6 ↗	6.2 ↗	8.0 ↑
1.8 ↗ (S)	3.1 ↗	4.6 ↗	6.2 ↑

```
Check: V(cell) = −1 + 0.9·V(best neighbour): 8.0 = −1 + 0.9·10, 6.2 = −1 + 0.9·8.0 ✓
```

Values are unique; optimal policies may not be — several monotone paths to G tie.

The cliff: SARSA vs Q-learning

Step −1, falling −100, both trained with the same ε-greedy exploration (ε = 0.1). **Q-learning hugs the cliff**: its max-target evaluates pure greedy play and ignores its own slips, so it learns the optimal-but-risky path and keeps falling during training (≈ −55 per episode while learning). **SARSA detours above the cliff**: it prices its own exploration slips into the values (≈ −24 while learning). Same Bellman equation, one symbol apart; decay ε → 0 and both become optimal.

The three dials

- **α (step size):** theory wants $\sum\alpha = \infty$, $\sum\alpha^2 < \infty$ (e.g. 1/n); practice: constant 0.05–0.5, or Adam in deep RL.
- **γ (horizon):** effective look-ahead ≈ 1/(1−γ); 0.99 ≈ 100 steps. Higher γ → bigger values → usually smaller α.
- **ε (exploration):** start ≈ 0.2–1.0, decay toward ≈ 0.01 (GLIE). Alternatives: UCB, softmax, entropy bonuses.

> Guarantee: tabular Q-learning → Q* with probability 1, if every (s,a) is visited infinitely often and α obeys Robbins–Monro.

8 · Scaling up: deep RL, policy gradients, RLHF

$$Q(s,a;\mathbf{w}), \qquad \mathbf{w} \leftarrow \mathbf{w} + \alpha\,\delta\,\nabla_{\mathbf{w}} Q(s,a;\mathbf{w})$$

Function approximation: similar states share estimates — generalisation. The price is the **deadly triad**: approximation + bootstrapping + off-policy learning together can diverge. Deep RL is engineering around this triad.

DQN (Mnih et al. 2015, Atari from pixels): regression toward a frozen bootstrap target $L(\mathbf{w}) = \mathbb{E}[(r + \gamma \max_{a'} Q(s', a'; \mathbf{w}^-) - Q(s,a;\mathbf{w}))^2]$, made stable by **experience replay** (breaks correlation, reuses data) and a **target network** $\mathbf{w}^-$ (stops the target chasing itself). Double DQN fixes max-overestimation; Rainbow stacks the tricks.

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi\left[\nabla_\theta \log \pi(A|S;\theta)\; G_t\right]$$

Policy gradient: make good actions more probable, in proportion to their return. Why: continuous actions (no argmax), stochastic policies for free, smooth updates. A baseline cuts variance without bias; an **actor–critic** lets the critic's TD error grade the actor. Lineage: REINFORCE → A2C/A3C → TRPO → PPO (powers RLHF) · SAC for continuous control.

The landscape. Model-free: value-based (DQN, Rainbow), policy gradient (REINFORCE, PPO, SAC), actor–critic (A3C, DDPG, TD3). Model-based: plan + learn (AlphaZero, MuZero), Dyna (real + imagined replay), world models (Dreamer). Model-based methods trade model error for sample efficiency.

RLHF — RL you used today: (1) supervised fine-tuning on human demonstrations; (2) humans rank answer pairs → train a reward model R(x,y); (3) PPO optimises the policy against R with a KL leash to the SFT model. Mapping: state = the conversation so far, action = the next token, policy = the language model, reward = learned preference, sparse, end-of-response. DPO folds step 3 into a direct loss; the framing survives.

Where RL runs the show: games and self-play (AlphaGo → MuZero), robotics (sim-to-real locomotion, manipulation), systems (data-centre cooling −40% energy, chip floorplanning, fusion plasma control), personalisation (bandits behind ads, feeds, recommendations). Common thread: cheap simulation or abundant interaction data.

Honest limits: sample hunger (DQN: ~200M frames per game); reward hacking (agents optimise the reward as written — the CoastRunners boat circles for points); sim-to-real gaps and non-stationarity; fragility to seeds and tuning. RLHF reward models are learned proxies — the same specification-gaming pressure applies to frontier LLMs. Reward design is alignment in miniature.

9 · The lecture in six lines

1. RL learns behaviour from evaluative, delayed feedback — no labels, only consequences.
2. The MDP ⟨S, A, P, R, γ⟩ formalises the problem; the Markov state carries the burden.
3. Bellman: value = reward + γ·next value; optimality replaces the average with a max.
4. DP plans with a model; MC averages real returns; TD bootstraps online — one template update.
5. SARSA learns the policy it runs; Q-learning learns the optimum it doesn't — cliff walking tells them apart.
6. Tables die at scale: function approximation → DQN, policy gradients, actor–critic, RLHF.

10 · Going deeper

- Sutton & Barto, *Reinforcement Learning: An Introduction* (2e) — free online; ch. 3–6 cover these notes.
- David Silver's RL lectures (UCL/DeepMind) — the same arc, with proofs.
- OpenAI, *Spinning Up in Deep RL* — from Q-learning to PPO with working code.
- Watkins & Dayan (1992), Q-learning. *Machine Learning* 8, 279–292. · Mnih et al. (2015), *Nature* 518. · Ouyang et al. (2022), InstructGPT/RLHF.